



WSQ

Lesson Plan

For

AI Vibe Coding for iOS Mobile Apps Development

TGS-2023037545

Software Design 3 · ICT-DES-3005-1.1

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN 201200696W

Version 1.2 · 12 September 2026

Document Version Control Record

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.2	12 September 2026	Added runnable XcodeGen projects to all Activities, three complete capstone apps, current Simulator verification, final app icons and a gated App Store Connect preparation workflow.	Tertiary Infotech Academy
1.1	12 September 2026	Added the end-to-end iOS vibe-coding prompt pack, per-Activity generation/review prompts, prompt slides, Apple implementation references and aligned slide citations.	Tertiary Infotech Academy
1.0	12 September 2026	Initial issue: portable C++ core, Objective-C++ adapter, SwiftUI shell, AI-assisted TDD, ten Activities and aligned assessments.	Tertiary Infotech Academy

Table of Contents

Section	Page
Course and Assessment Parameters	3
Learning Outcomes and Slide Alignment	3
Day 1 · Requirements, Core and Tooling	4
Day 2 · Shell, Interoperability and Evidence	5
Activity Alignment	5–6
Facilitation and Evidence Notes	6

Course and Assessment Parameters

Item	Approved value
Course	AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)
TSC	Software Design 3 · ICT-DES-3005-1.1
Duration	2 days · 16 contact hours
Instruction	7 hours lecture/didactic/demonstration · 7 hours practical
Assessment	WA (SAQ) 1 hour · PP 1 hour

Learning Outcomes and Slide Alignment

Outcome	Criteria	Deck slides
LO1: Determine basic software components using C++ methodologies to meet functional specifications.	K1, A1	21–66
LO2: Apply C++ methodologies and tools for software creation.	K2, A2	67–113
LO3: Select essential C++ controls and features to meet software design requirements.	K3, A3	114–159
LO4: Examine the interoperability and functionality of C++ software components.	K4, A4	160–206
LO5: Generate C++ design documentation aligned with user specifications.	K5, A5	207–252

Day 1 · Requirements, Core and Tooling

Time	Content	Method	Deck reference	Hours
09:30–11:00	Requirements briefing, component boundary and Activity 1	Facilitated analysis · demonstration · practical	Slides 21–60	1.5
11:00–13:00	Portable C++ design and Activity 2	Demonstration · guided practical	Slides 61–65	2.0
13:00–14:00	Lunch	—	—	0
14:00–16:00	C++ methods, CMake and AI-assisted TDD	Lecture · demonstration · questioning	Slides 67–102	2.0
16:00–17:15	Activity 3: AI-Assisted TDD	Practical · peer review	Slides 103–107	1.25
17:15–18:30	Activity 4: Toolchain and Quality Gates · Day review	Practical · retrieval practice	Slides 108–112	1.25

Day 1 contact total: 8.0 hours.

Day 2 · Shell, Interoperability and Evidence

Time	Content	Method	Deck reference	Hours
09:30–11:15	SwiftUI controls, state and Objective-C++ boundary	Lecture · demonstration	Slides 114–148	1.75
11:15–13:00	Activities 5–6: Adapter and SwiftUI controls	Guided practical	Slides 149–158	1.75
13:00–14:00	Lunch	—	—	0
14:00–15:20	Activities 7–8: Persistence and interoperability evidence	Practical · assessment of evidence	Slides 196–205	1.33
15:20–16:30	Activities 9–10: Documentation and capstone evidence	Practical · facilitated review	Slides 242–251	1.17
16:30–17:30	Written Assessment (SAQ)	Individual assessment	Closing administration slides 257–260	1.0
17:30–18:30	Practical Performance	Individual assessment	Assessment papers and Activity evidence	1.0

Day 2 contact total: 8.0 hours. Course total: 16.0 hours, comprising 14 training hours and 2 assessment hours.

Activity Alignment

Activity	Mapping	Slides	Output
Activity 1: Requirements and Component Boundary	K1, A1	56–60	Requirement matrix, boundary diagram and acceptance criteria
Activity 2: Portable C++ Domain Core	K1, A1	61–65	Compiling C++20 library and domain tests
Activity 3: AI-Assisted TDD for C++	K2, A2	103–107	Prompt contract, failing trace, minimal patch and passing trace
Activity 4: Toolchain and Quality Gates	K2, A2	108–112	Warning-clean build, quality script and evidence log
Activity 5: Objective-C++ Adapter	K3, A3	149–153	Bridge header, .mm adapter and round-trip test
Activity 6: SwiftUI Controls and State	K3, A3	154–158	SwiftUI entry flow, deterministic previews and validation states

Activity	Mapping	Slides	Output
Activity 7: Persistence and Error Recovery	K4, A4	196–200	Versioned JSON repository, migration fixture and recovery tests
Activity 8: Interoperability Validation Scorecard	K4, A4	201–205	Test matrix, execution evidence and release scorecard
Activity 9: Architecture and Traceability Documentation	K5, A5	242–246	C4 diagrams, two ADRs, API contract and traceability matrix
Activity 10: Capstone: BudgetBuddy Release Evidence	A1, A2, A3, A4, A5	247–251	Working app, complete tests, architecture pack and deployment checklist

Facilitation and Evidence Notes

The trainer demonstrates mechanisms and design judgement in the deck. Each Activity also has a readable copy/paste code-generation prompt slide, while the full generation and review/repair prompts remain in the Learner Guide and Activity folders. No click-by-click Xcode procedure is placed in the deck. Each practical segment ends with an observable acceptance criterion and retained evidence.

Assessment briefing precedes assessment. WA tests K1–K5 through five open-ended questions; PP tests A1–A5 through three coherent tasks based on Activities 1–10.